🎯 Lidar Tracking

1. Course Content

The A1 Pro robot uses lidar to scan the front environment in real-time, identifies the nearest target, and uses a dual-loop PID control algorithm to drive the Ackermann chassis to maintain a specified safe distance and follow the target while aligning its heading.

---

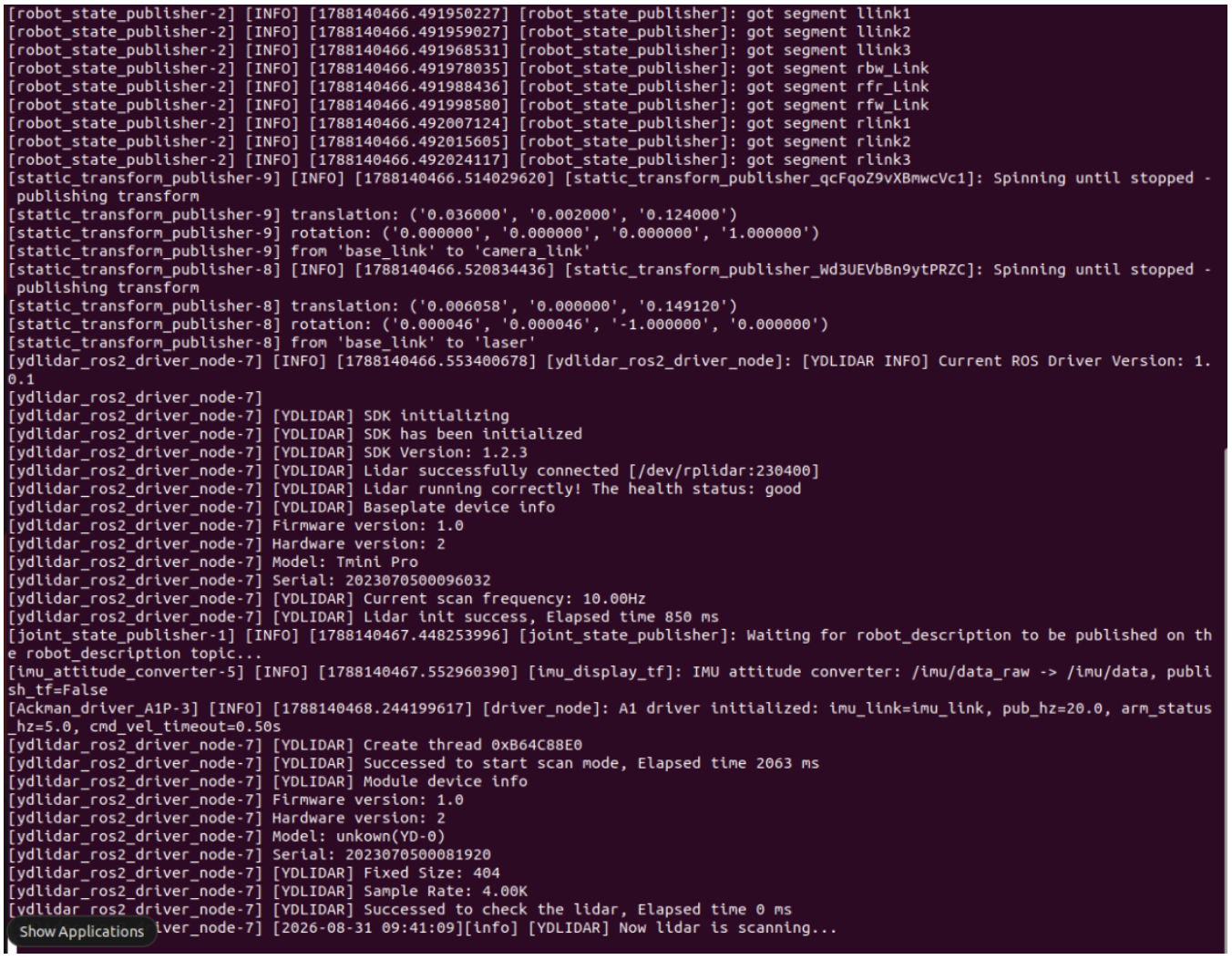
2. Preparation and Running

This example requires opening 2 terminals to start the following nodes in sequence:

2.1 Start Chassis + Lidar

Run in **Terminal 1**:

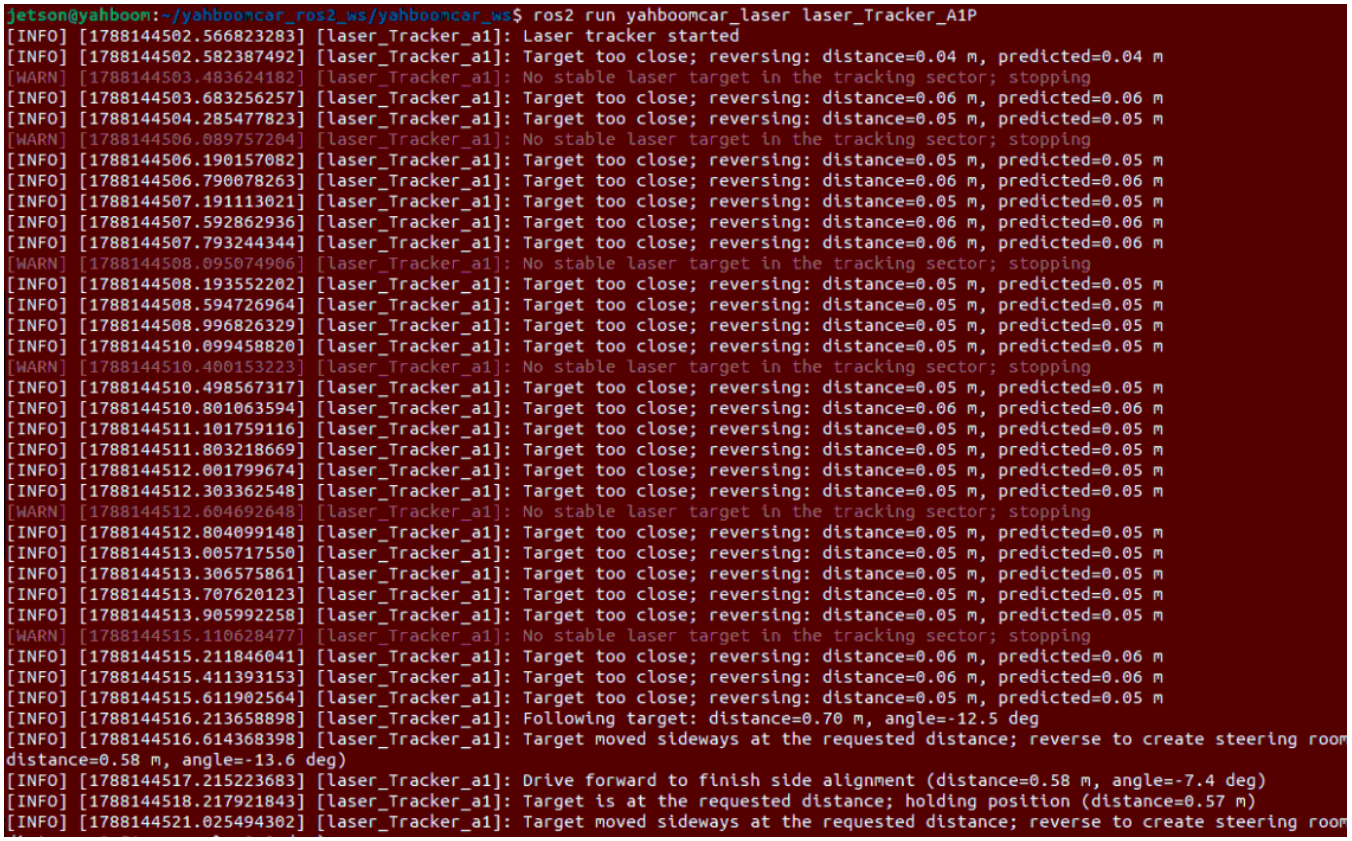
```
ros2 launch yahboomcar_nav laser_bringup_launch.py
```



2.2 Start Lidar Tracking Program

Run in **Terminal 2**:

```
ros2 run yahboomcar_laser laser_Tracker_A1P
```



3. Tracking Logic and Dynamic Parameter Adjustment

3.1 Tracking Motion Logic and Effects

Place an object that can be scanned by the lidar in front of the robot (such as cardboard, palm, or obstacle):

1. **Distance Following**: Move the object slowly forward and backward, and the robot will automatically move forward or backward to maintain the preset safe distance of approximately `0.55m` from the target.
2. **Heading Alignment**: Move the object slowly left and right, and the robot will automatically adjust the Ackermann steering servo so that the center of the chassis front faces the target.
3. **Safety Protection**: When the target moves out of the lidar's valid detection sector, the robot automatically stops moving.



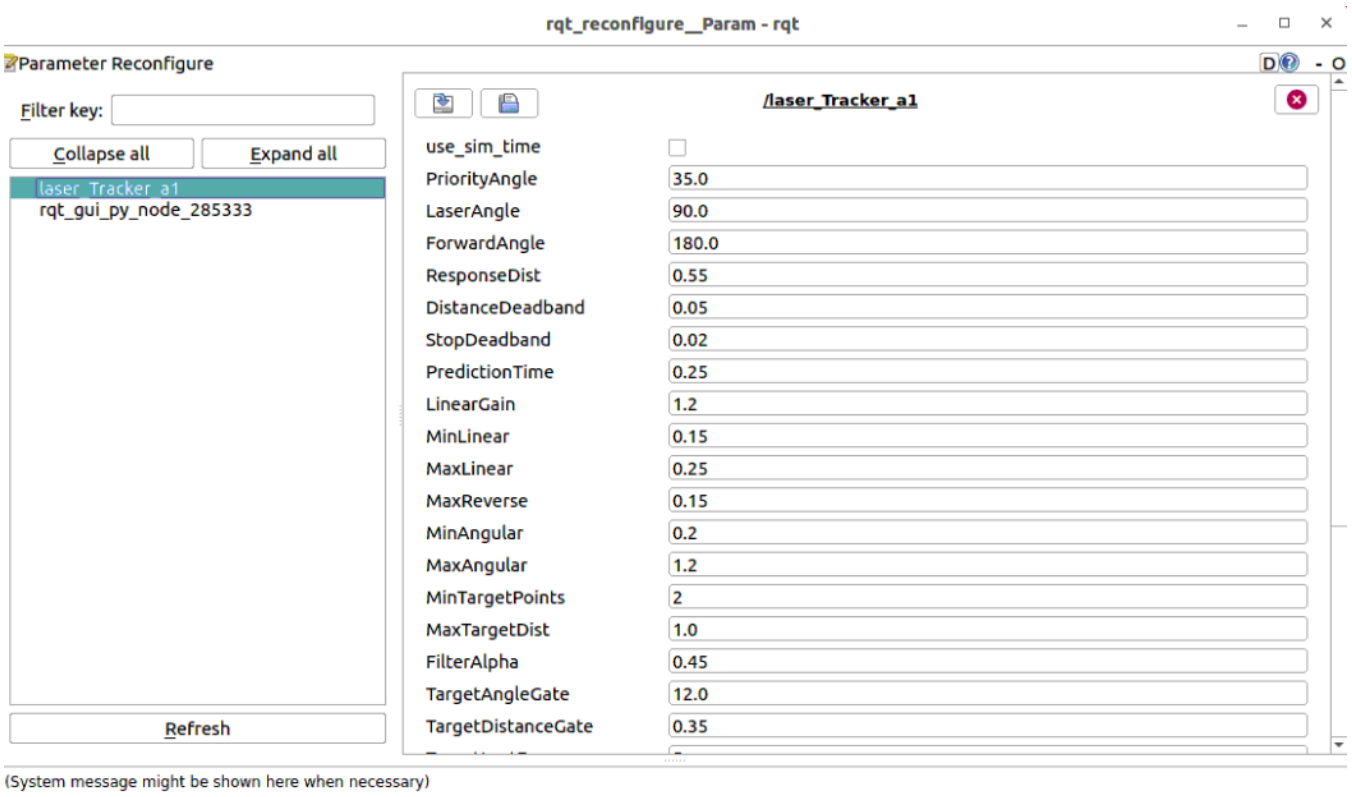
> **Note**
>
> To end the tracking example, press `Ctrl+C` in each terminal window to stop the program.

3.2 Dynamic Parameter Adjustment (rqt_reconfigure)

During the lidar tracking program operation, you can use ROS 2's dynamic parameter adjuster (`rqt_reconfigure`) to adjust tracking safe distance, dead zone range, forward/reverse maximum speed, and prediction filter parameters in real-time. Changes take effect immediately.

Run the following command in a **new terminal** to start the dynamic parameter adjuster:

```
ros2 run rqt_reconfigure rqt_reconfigure
```



Select the `laser_Tracker_a1` node on the left side of the opened panel, then use the visual sliders on the right side to dynamically modify various control parameters.

3.3 Core Parameter Description

Parameter Name	Default Value	Description
`ResponseDist`	`0.55`	Standard following safe distance to maintain with target (meters)
`PriorityAngle`	`35.0`	Half-angle of the high-priority search sector directly ahead (degrees)
`LaserAngle`	`90.0`	Half-angle of the wide-angle target capture sector (degrees)
`DistanceDeadband`	`0.05`	Startup distance dead zone threshold for triggering chassis motion (meters)
`StopDeadband`	`0.02`	Stop distance dead zone threshold for maintaining stationary state (meters)
`PredictionTime`	`0.25`	Forward prediction time compensation for target distance (seconds)

Parameter Name	Default Value	Description
`LinearGain`	`1.20`	Distance error linear velocity proportional gain coefficient
`MinLinear` / `MaxLinear`	`0.15` / `0.25`	Following forward linear velocity range (m/s)
`MaxReverse`	`0.15`	Maximum reverse linear velocity when target is too close (m/s)
`MinAngular` / `MaxAngular`	`0.20` / `1.20`	Ackermann steering angular velocity range (rad/s)
`MinTargetPoints`	`2`	Lower limit of clustering points for determining valid target
`FilterAlpha`	`0.45`	Low-pass smoothing filter coefficient for target position
`SideAlignEnabled`	`True`	Side target reverse turning smooth alignment function switch
`AllowReverse`	`True`	Whether to allow reverse to maintain safe distance (`True` / `False`)
`Switch`	`False`	Main switch for tracking function (`True` = pause, `False` = run)

4. Core Source Code and Algorithm Analysis

Source Code Path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_laser/yahboomcar_laser/laser_Tracker_A1P.py
```

4.1 Point Cloud Clustering and Spatial Gate Continuous Tracking

For discrete laser noise and background clutter, the algorithm uses `find_tracking_target` to perform proximity clustering on the forward sector point cloud, extracting the geometric center angle and distance median of the object. When a target is locked, **angle gate (`TargetAngleGate`)** and **distance gate (`TargetDistanceGate`)** are introduced for continuous tracking matching, preventing the robot from mistakenly following background moving objects:

```
    # Extract forward sector laser points with chassis front as 0°
    points = [
        point for point in iter_scan_points(scan_data, self.ForwardAngle)
        if abs(point[0]) <= 90.0
    ]
    # Point cloud clustering and gate association matching target
    target = find_tracking_target(
        points,
```

```
        laser_angle=laser_angle,
        priority_angle=priority_angle,
        min_points=self.MinTargetPoints,
        max_distance=self.MaxTargetDist,
        max_angle_gap=max_angle_gap,
        previous_target=self.filtered_target,
        angle_gate=self.TargetAngleGate,
        distance_gate=self.TargetDistanceGate,
    )
    if target is None:
        self._handle_missing_target()
        return
```

4.2 Target First-Order Smoothing, Forward Prediction and Dual Dead Zone Control

To ensure smooth and fluid motion of the Ackermann chassis, the algorithm applies low-pass smoothing filtering to the target position, calculates the target relative motion rate `range_rate` and performs forward time prediction (`PredictionTime`), combined with **startup dead zone (`DistanceDeadband`)** and **stop dead zone (`StopDeadband`)** dual judgment to eliminate frequent start-stop oscillation:

```
    # 1. Target smoothing filter and distance change rate calculation
    target_angle, target_distance = self._filter_target(target, now)
    # 2. Forward distance dynamic prediction: compensate for robot acceleration/deceleration
lag in advance
    predicted_distance = max(0.0, target_distance + self.range_rate * self.PredictionTime)
    actual_error = measured_distance - self.ResponseDist
    predicted_error = predicted_distance - self.ResponseDist

    # 3. Dynamic state machine update for forward/reverse/stationary motion direction
    direction_error = select_tracking_direction_error(
        self.motion_direction, actual_error, predicted_error)
    self.motion_direction = update_tracking_motion_direction(
        self.motion_direction, direction_error,
        self.DistanceDeadband, self.StopDeadband,
        self.AllowReverse, reverse_blocked=False,
    )
    # 4. Calculate forward/reverse linear velocity
    linear = compute_tracking_linear_speed(
        predicted_error=predicted_error,
        direction=self.motion_direction,
        gain=self.LinearGain,
        minimum_speed=self.MinLinear,
        maximum_speed=(self.MaxReverse if self.motion_direction < 0 else self.MaxLinear),
    )
```

4.3 Ackermann Side Alignment and Parking Fine-Tuning Mechanism

Since the Ackermann chassis cannot rotate in place, when a close-range target is at a large side angle, the algorithm automatically triggers the `SideAlignEnabled` two-stage parking alignment: first back up slightly to create turning space, then use front-wheel steering to move forward while aligning with the target center.